

Project Requirements

Kogs

Color Palette Generator

Frontend: Vue

CSS: Tailwind

Backend: Laravel

Database: MySQL

01 — Project Requirements

1	User can upload an image and extract dominant colors from it
2	User can search by keyword or mood to generate a color palette
3	Color frequency/clustering algorithm to extract at least 5 dominant colors per palette
4	Display each color with its HEX, RGB, and HSL values
5	User can name and save palettes to their account
6	Copy HEX code to clipboard with one click
7	Export palette as a PNG image
8	Palette gallery page showing all saved palettes
9	User authentication — register and login
10	Vue + Tailwind frontend running locally via Vite dev server
11	Laravel REST API backend running locally via php artisan serve
12	MySQL database running locally via XAMPP or Laragon
13	External API: The Color API or Unsplash API
14	GitHub repo with proper README and consistent commits throughout the month

02 — 1-Month Roadmap

Week 1 — Setup & Planning

- Set up GitHub repo and push initial commit
- Initialize Vue project with Vite and install Tailwind CSS
- Scaffold Laravel project and configure .env for local MySQL connection
- Set up MySQL database locally using XAMPP or Laragon
- Get API key for Unsplash or The Color API and test it in Postman
- Install Intervention Image package in Laravel for image processing

Week 2 — Core Features

- Build image upload UI in Vue using a file input and preview display
- Send uploaded image to Laravel backend via FormData and axios
- Implement color extraction logic in Laravel using Intervention Image
- Return dominant colors as HEX, RGB, and HSL values to the frontend
- Build keyword/mood search that calls The Color API or Unsplash and returns palette
- Set up Laravel auth routes — register, login, logout using Sanctum

Week 3 — Algorithm + Polish

- Finalize the color frequency/clustering algorithm — ensure at least 5 colors returned
- Build the palette display UI showing color swatches with HEX, RGB, HSL labels
- Implement copy HEX to clipboard on click using the Clipboard API in Vue
- Implement export palette as PNG using HTML Canvas in the frontend
- Save and retrieve palettes from MySQL via Laravel API
- Build palette gallery page and clean up UI with Tailwind

Week 4 — Finalize & Testing

- Fix bugs found during testing
- Handle edge cases — no image selected, invalid file type, API failures
- Write the GitHub README — project overview, setup steps, and screenshots
- Do a full end-to-end test of all features locally
- Final code cleanup and ensure all commits are meaningful

03 — Tech Stack Setup Guide

Vue (Frontend Framework)

1. Install Node.js first, then run: `npm create vue@latest` to scaffold the project
2. Learn components, props, and emits first — then Vue Router for page navigation
3. For file uploads, learn how to use FormData and send it via axios with multipart/form-data header
4. Use Pinia to store the logged-in user and the current palette state globally
5. Learn how to use the Clipboard API (`navigator.clipboard.writeText`) for copy-to-clipboard feature

Tailwind CSS

1. Install via npm: `npm install -D tailwindcss` and follow the Vue setup guide on tailwindcss.com
2. Focus on grid layouts for the palette gallery — use `grid-cols` and `gap` utilities
3. Use `aspect-ratio` and `object-cover` for displaying uploaded image previews cleanly

4. Use the Tailwind Playground (play.tailwindcss.com) to test classes before writing code

Laravel (Backend)

1. Install Composer first, then run: `composer create-project laravel/laravel palette-api`
2. Set up your `.env` file: `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD` pointing to your local MySQL
3. Run `php artisan migrate` to create tables, `php artisan serve` to start the local server
4. Install Intervention Image: `composer require intervention/image` — used to read pixel data from uploads
5. Use Laravel Sanctum for API authentication — follow the official docs for setup
6. Test all API endpoints in Postman before connecting to Vue frontend

MySQL (Database)

1. Install XAMPP or Laragon to run MySQL locally — start MySQL from the control panel
2. Use phpMyAdmin or TablePlus to manage your tables visually
3. Your main tables: `users`, `palettes`, `palette_colors` (one palette has many colors)
4. Learn Eloquent one-to-many relationships — a palette hasMany colors
5. Use Laravel migrations to manage your schema — never edit the DB manually